

SDA-PRO: Security Detection and Automated Response Platform

Software Design and Architecture Project Report

Project Title: SDA-PRO – Security Detection and Automated Response Platform

Course: Software Design and Architecture

Submitted By: Muhammad Jaffar Nawaz-56474 || Mustafa Tahir-56462 || Ibad ur Rehman-56468

Instructor: Sir Uzair Rasheed

Department: Software Engineering

University: Riphah International University

GITHUB REPO LINK: <https://github.com/56474-MJN/SDA-PROJECT.git>

DECLARATION

I hereby declare that this project entitled “SDA-PRO: Security Detection and Automated Response Platform” is my own work. The project has been developed in accordance with the requirements of the Software Design and Architecture course and has not been submitted elsewhere for academic credit.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my course instructor for providing guidance throughout the development of this project. I also thank my department and university for providing the resources necessary to complete this work.

ABSTRACT

SDA-PRO is a Security Detection and Automated Response Platform designed to demonstrate the application of Software Design and Architecture principles in a cybersecurity environment. The system simulates the ingestion of security alerts from multiple sources, enriches them with contextual information, creates incidents, manages incident lifecycles, and automates response actions.

The project follows a layered architecture and incorporates MVC principles. Multiple design patterns including Factory Method, Abstract Factory, Singleton, Strategy, State, Observer, Chain of Responsibility, Decorator, Composite, Adapter, Proxy, and Facade were implemented to ensure modularity, maintainability, and extensibility.

The platform also includes a Java Swing graphical dashboard, external integration simulation modules, UML documentation, testing support, and deployment artifacts.

Keywords: Cybersecurity, Incident Response, Design Patterns, MVC, SOA, Java.

TABLE OF CONTENTS

SDA-PRO: Security Detection and Automated Response Platform	1
Software Design and Architecture Project Report	1
COVER PAGE	1
DECLARATION	1
ACKNOWLEDGEMENT	2
ABSTRACT	2
TABLE OF CONTENTS	2
CHAPTER 1 – INTRODUCTION	3
1.1 Background	3
1.2 Problem Statement	3
1.3 Project Objectives	4
1.4 Scope	4
CHAPTER 2 – SYSTEM ARCHITECTURE	4
2.1 Architectural Overview	4
Component Diagram	5
2.2 MVC Architecture	6
Model	6
View	6
Controller	7
CHAPTER 3 – UML DESIGN	7
3.1 Alert Processing Diagram	8
Patterns Shown	8
3.2 Incident Lifecycle Diagram	8
Patterns Shown	9
3.3 Response Engine Diagram	10

3.4 Enrichment Pipeline Diagram	10 Patterns
Shown	10
3.5 Event System Diagram	11
CHAPTER 4 – DESIGN PATTERN IMPLEMENTATION	12
4.1 Factory Method Pattern	12
4.2 Abstract Factory Pattern.....	12
4.3 Singleton Pattern	12
4.4 Strategy Pattern	13
4.5 State Pattern	13
4.6 Observer Pattern	13
4.7 Chain of Responsibility Pattern	14
4.8 Decorator Pattern	14
4.9 Composite Pattern.....	14
4.10 Adapter Pattern	15
4.11 Proxy Pattern	15
4.12 Facade Pattern	15
CHAPTER 5 – SEQUENCE DIAGRAMS	16
5.1 Alert Ingestion Workflow	16
CHAPTER 6 – GUI IMPLEMENTATION	17
CHAPTER 7 – EXTERNAL INTEGRATIONS	20
CHAPTER 8 – TESTING AND VALIDATION.....	21
CHAPTER 9 – DEPLOYMENT	22
9.1 Maven Build	23
9.2 Repository Layer	23
9.3 ADR Documents	23
CHAPTER 10 – RESULTS AND DISCUSSION	24
CHAPTER 11 – CONCLUSION	24
APPENDICES	25

CHAPTER 1 – INTRODUCTION

1.1 Background

Modern organizations generate large volumes of security alerts from firewalls, endpoint detection systems, SIEM platforms, and cloud security services. Security Operations Centers (SOCs) must process these alerts efficiently to detect and respond to threats.

Manual handling of alerts can lead to delayed responses, missed incidents, and inconsistent decision-making. Therefore, automated incident management platforms have become essential.

1.2 Problem Statement

Organizations face the following challenges:

- Excessive alert volume
- Alert fatigue
- Slow incident response

- Inconsistent threat analysis
- Limited integration between security tools

A system is required to automate alert processing and incident response while maintaining flexibility and extensibility.

1.3 Project Objectives

The objectives of SDA-PRO are:

- Ingest security alerts
- Normalize alert formats
- Correlate related events
- Create incidents automatically
- Execute response strategies
- Demonstrate software architecture concepts
- Implement multiple design patterns
- Provide a graphical dashboard

1.4 Scope

The system focuses on simulation and architectural implementation rather than production deployment.

CHAPTER 2 – SYSTEM ARCHITECTURE

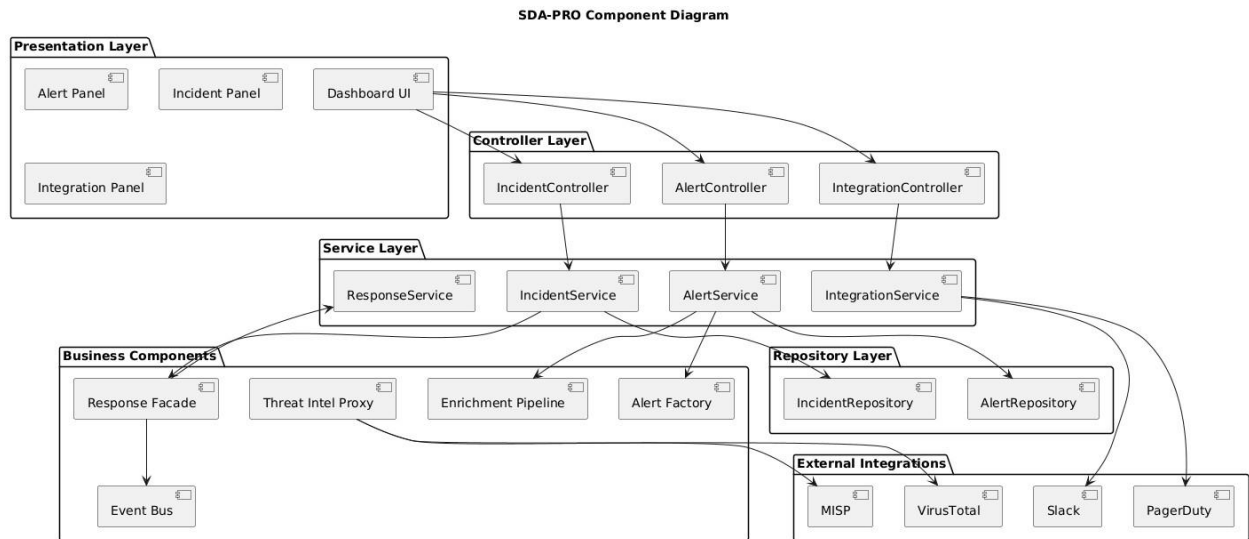
2.1 Architectural Overview

The system follows a layered architecture consisting of:

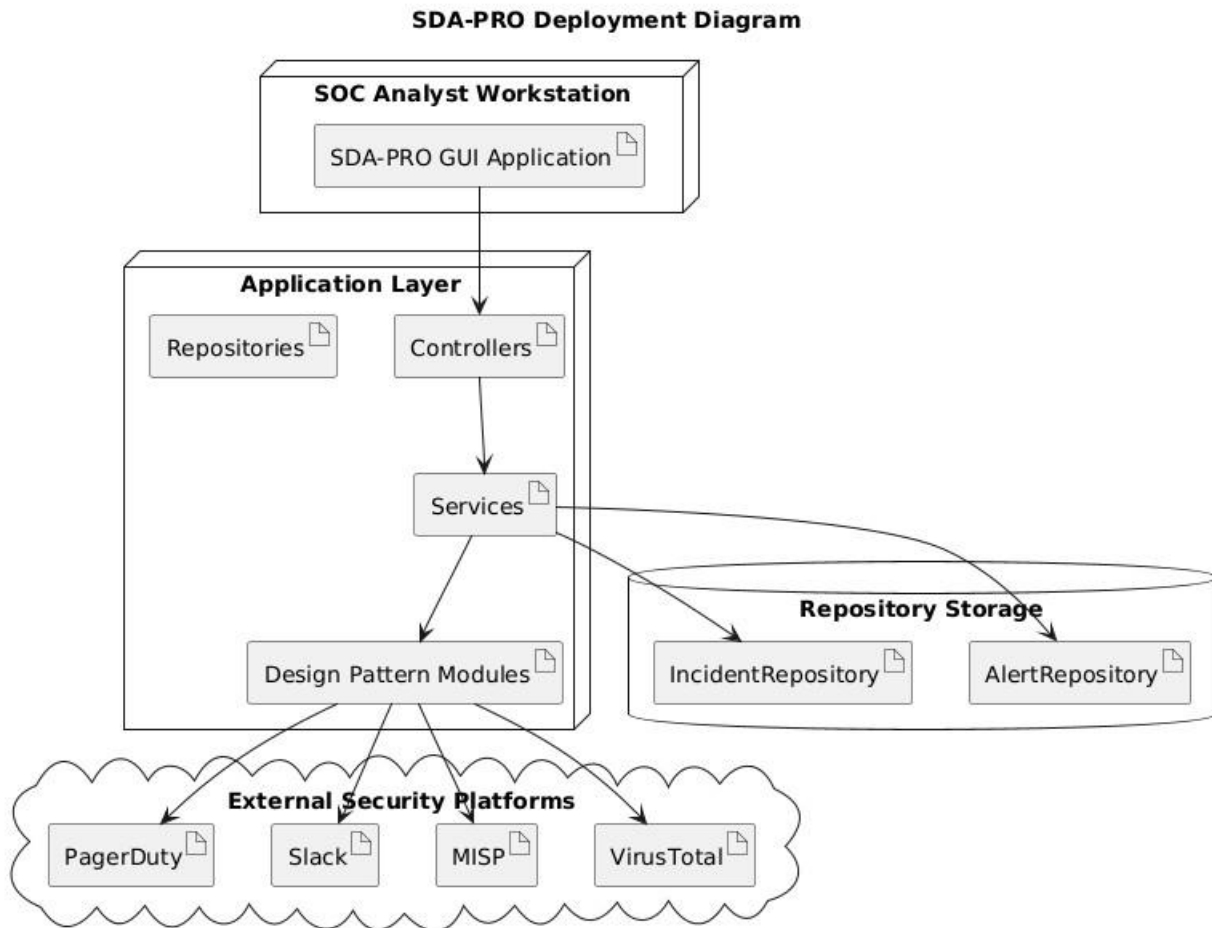
- Presentation Layer
- Controller Layer
- Service Layer
- Repository Layer
- Integration Layer

This separation ensures low coupling and high cohesion.

Component Diagram



Deployment Diagram



2.2 MVC Architecture

The application follows the Model-View-Controller pattern.

Model

Contains:

- Alert
- Incident
- AlertCampaign
- SingleAlert

View

Contains:

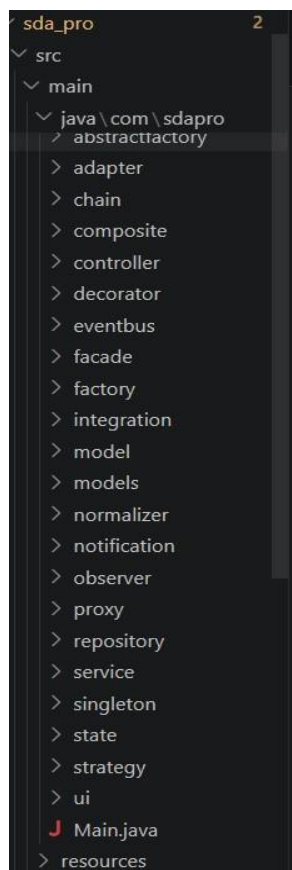
- Dashboard UI
- Alert View
- Incident View

Controller

Contains:

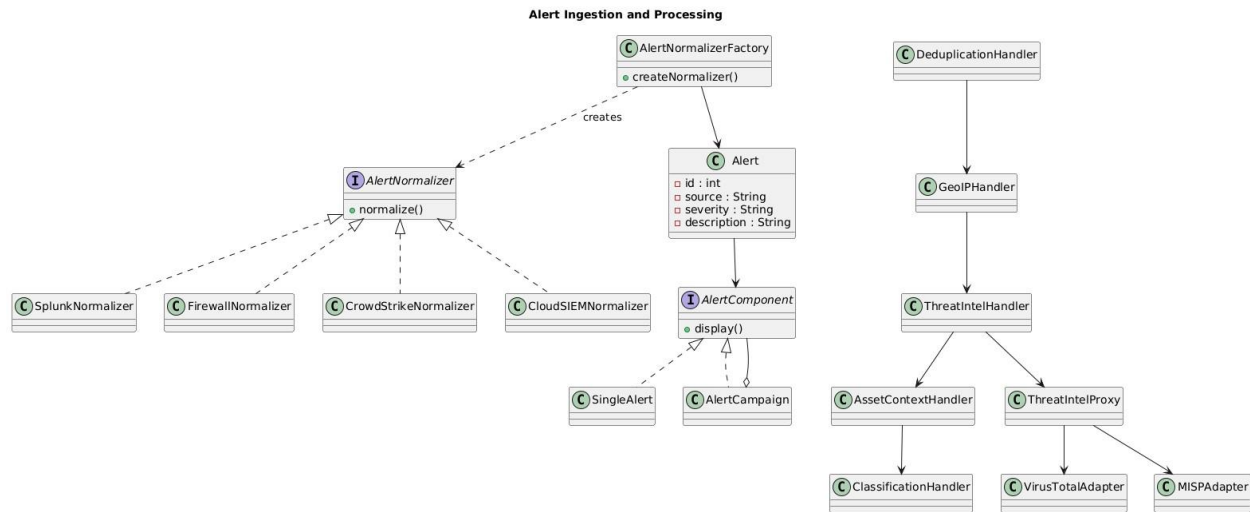
- AlertController
- IncidentController
- IntegrationController

Project Package Structure



CHAPTER 3 – UML DESIGN

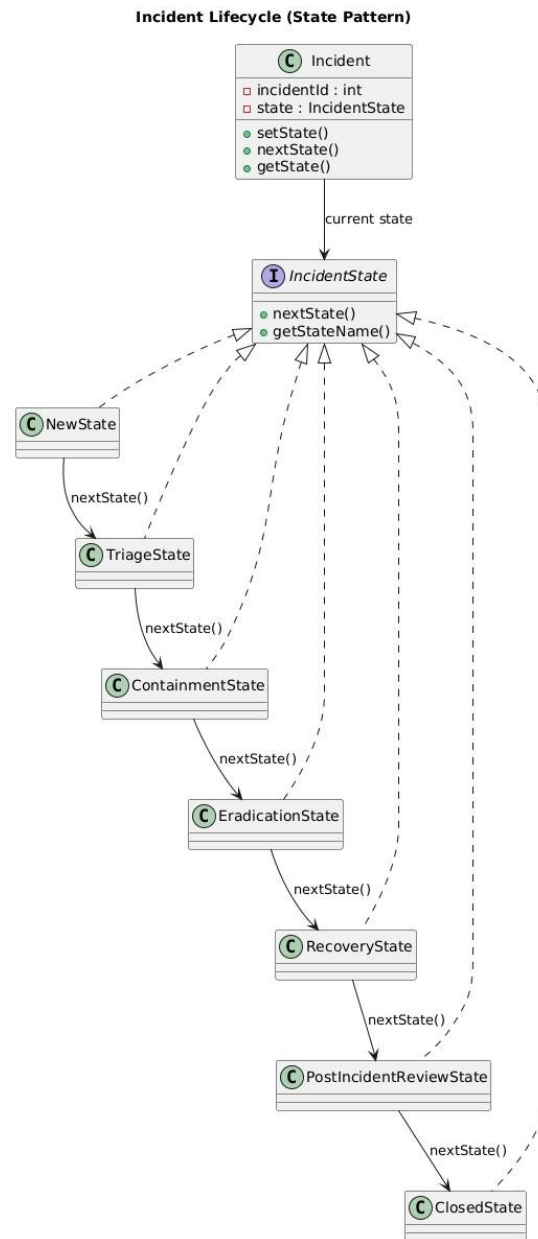
3.1 Alert Processing Diagram



Patterns Shown

- Factory Method → **AlertNormalizerFactory**
- Composite → **AlertComponent**, **SingleAlert**, **AlertCampaign**
- Chain of Responsibility → Enrichment Handlers • Proxy → **ThreatIntelProxy**
- Adapter → **VirusTotalAdapter**, **MISPAdapter**

3.2 Incident Lifecycle Diagram



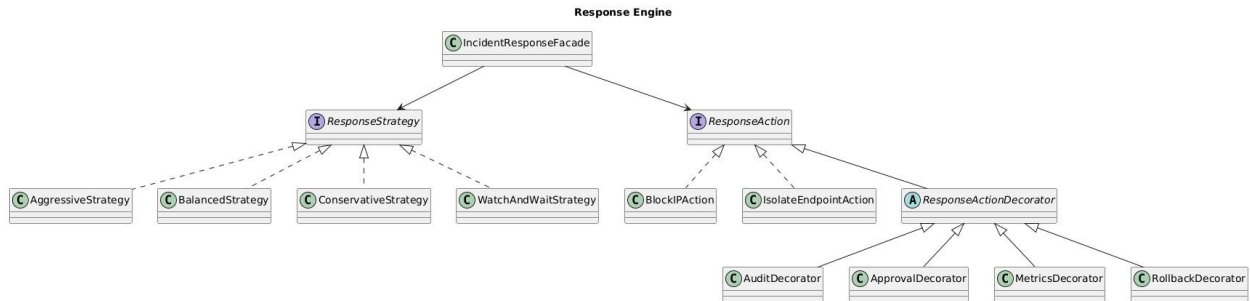
Patterns Shown

State Pattern → `IncidentState`, `NewState`, `TriageState`, `ContainmentState`, `EradicationState`, `RecoveryState`, `PostIncidentReviewState`, `ClosedState`

Context Class → `IncidentContext`

State Transitions → New → Triage → Containment → Eradication → Recovery → Post-Incident Review → Closed

3.3 Response Engine Diagram



Patterns Shown

Strategy Pattern → ResponseStrategy, AggressiveStrategy, BalancedStrategy, ConservativeStrategy, WatchAndWaitStrategy

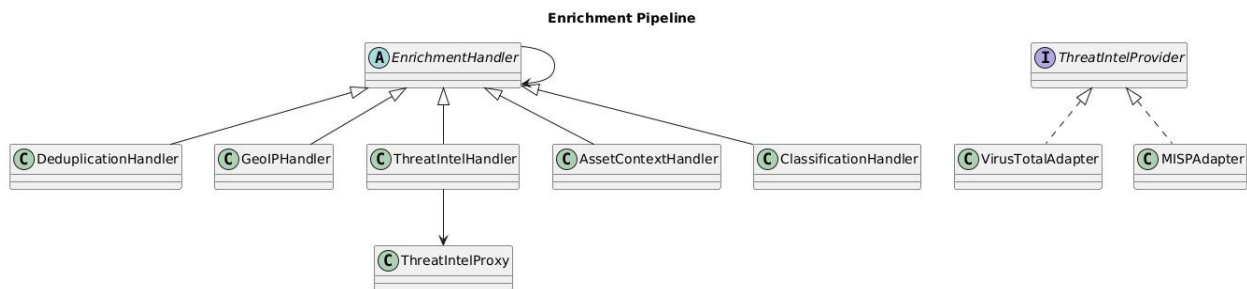
Decorator Pattern → AuditDecorator, ApprovalDecorator, MetricsDecorator, RollbackDecorator

Facade Pattern → IncidentResponseFacade

Response Service → ResponseService

Action Interface → ResponseAction

3.4 Enrichment Pipeline Diagram



Patterns Shown

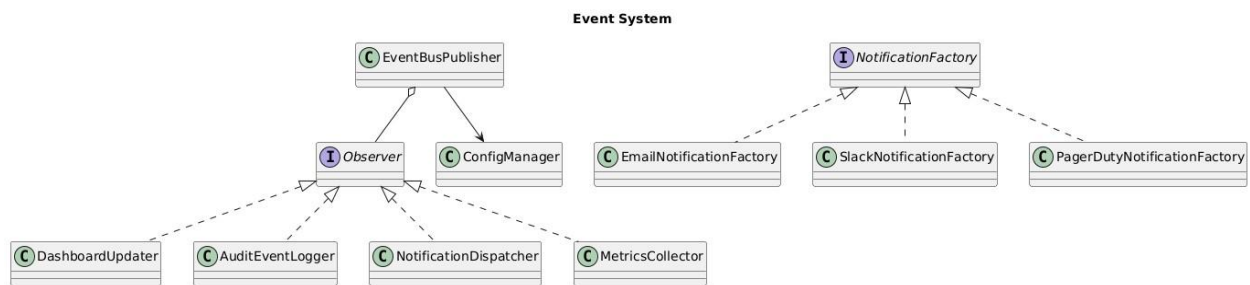
Chain of Responsibility → DeduplicationHandler, GeoIPHandler, ThreatIntelHandler, AssetContextHandler, ClassificationHandler

Proxy Pattern → ThreatIntelProxy

Adapter Pattern → VirusTotalAdapter, MISPAdapter

Alert Processing Flow → Deduplication → GeoIP → Threat Intelligence → Asset Context → Classification

3.5 Event System Diagram



Observer-based event processing.

Patterns Shown

Observer Pattern → DashboardUpdater, AuditEventLogger, NotificationDispatcher, MetricsCollector

Singleton Pattern → EventBusPublisher

Abstract Factory Pattern → EmailNotificationFactory, SlackNotificationFactory, PagerDutyNotificationFactory

Notification Products → EmailNotifier, SlackNotifier, PagerDutyNotifier

Event Distribution Flow → EventBusPublisher → Dashboard Updates → Audit Logging → Notifications → Metrics Collection

CHAPTER 4 – DESIGN PATTERN IMPLEMENTATION

4.1 Factory Method Pattern

Purpose:

Create alerts and normalizers dynamically.

Classes:

- AlertFactory
- AlertNormalizerFactory Benefits:

Encapsulation of object creation.

4.2 Abstract Factory Pattern

Classes:

- NotificationFactory
- EmailNotificationFactory
- SlackNotificationFactory
- PagerDutyNotificationFactory Purpose:

Create families of notification objects.

4.3 Singleton Pattern

Classes:

-
-

ConfigManager

EventBusPublisher Purpose:

Provide a single global instance.

4.4 Strategy Pattern

Classes:

- AggressiveStrategy
- BalancedStrategy
- ConservativeStrategy
- WatchAndWaitStrategy Purpose:

Dynamic response selection.

4.5 State Pattern

States:

- New
- Triage
- Containment
- Eradication
- Recovery
- PostIncidentReview
- Closed

Purpose:

Manage incident lifecycle.

4.6 Observer Pattern

Classes:

DashboardUpdater

-
-
- AuditEventLogger
- NotificationDispatcher
- MetricsCollector Purpose:

Event-driven communication.

4.7 Chain of Responsibility Pattern

Handlers:

- DeduplicationHandler
- GeoIPHandler
- ThreatIntelHandler
- AssetContextHandler
- ClassificationHandler Purpose:

Sequential alert enrichment.

4.8 Decorator Pattern

Decorators:

- AuditDecorator
- ApprovalDecorator
- MetricsDecorator
- RollbackDecorator

Purpose:

Extend response actions dynamically.

4.9 Composite Pattern

Classes:

AlertCampaign SingleAlert

-
-

Purpose:

Represent alert hierarchies.

4.10 Adapter Pattern

Adapters:

- VirusTotalAdapter
- MISPIAdapter
- SlackAdapter
- PagerDutyAdapter

Purpose:

Standardize external integrations.

4.11 Proxy Pattern

Class:

- ThreatIntelProxy

Purpose:

Control access to threat intelligence resources.

4.12 Facade Pattern

Class:

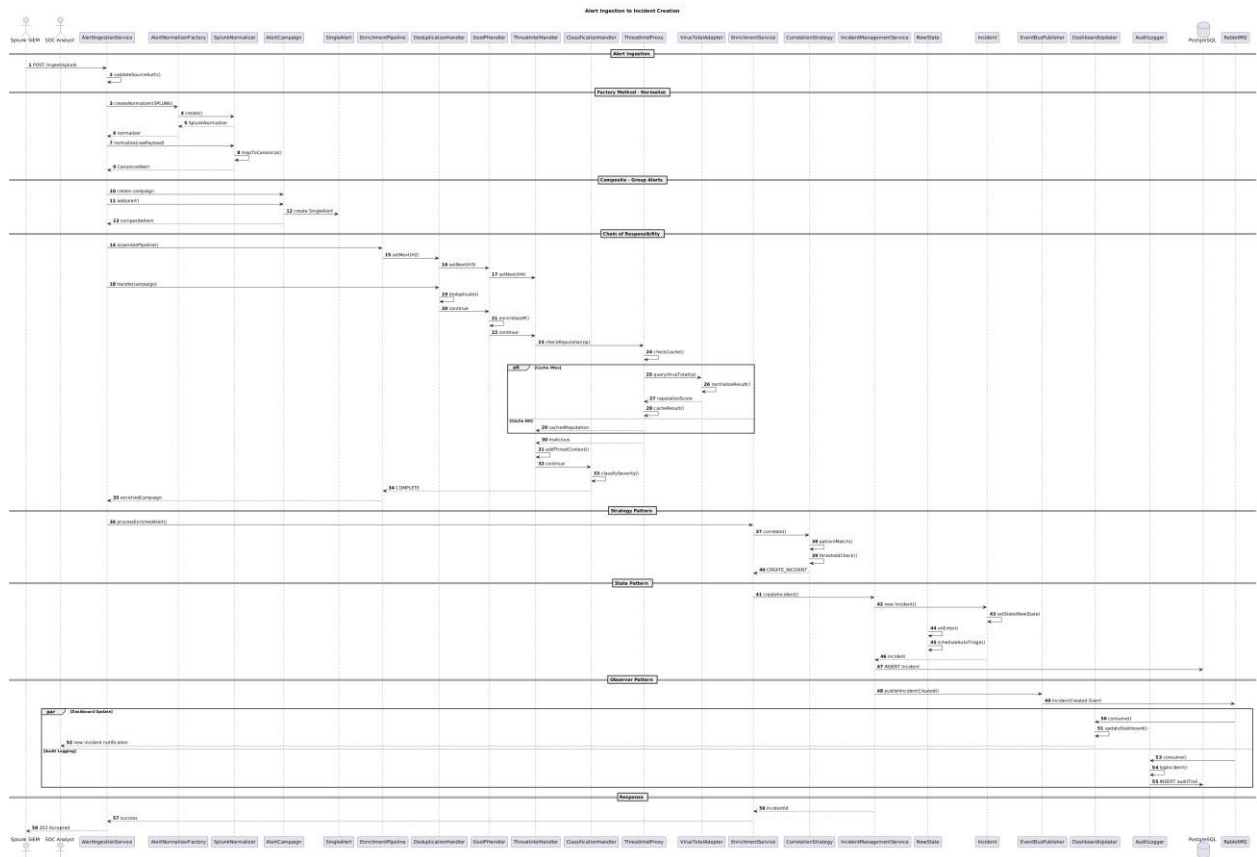
- IncidentResponseFacade

Purpose:

Provide a unified interface to incident response operations.

CHAPTER 5 – SEQUENCE DIAGRAMS

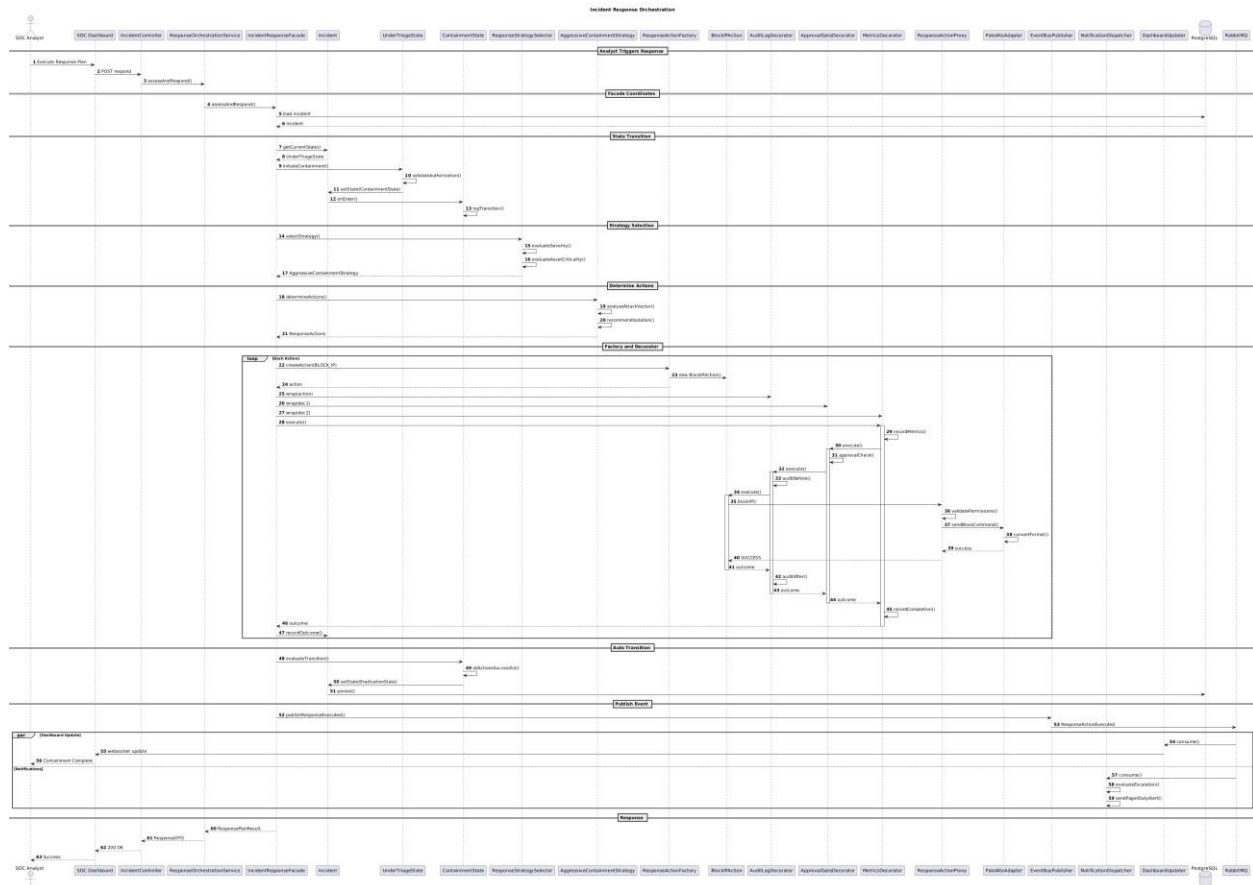
5.1 Alert Ingestion Workflow



Description:

Alert → Normalization → Enrichment → Correlation → Incident Creation → Event Publication

5.2 Incident Response Workflow



Description:

Analyst → Response Selection → Action Execution → Notifications

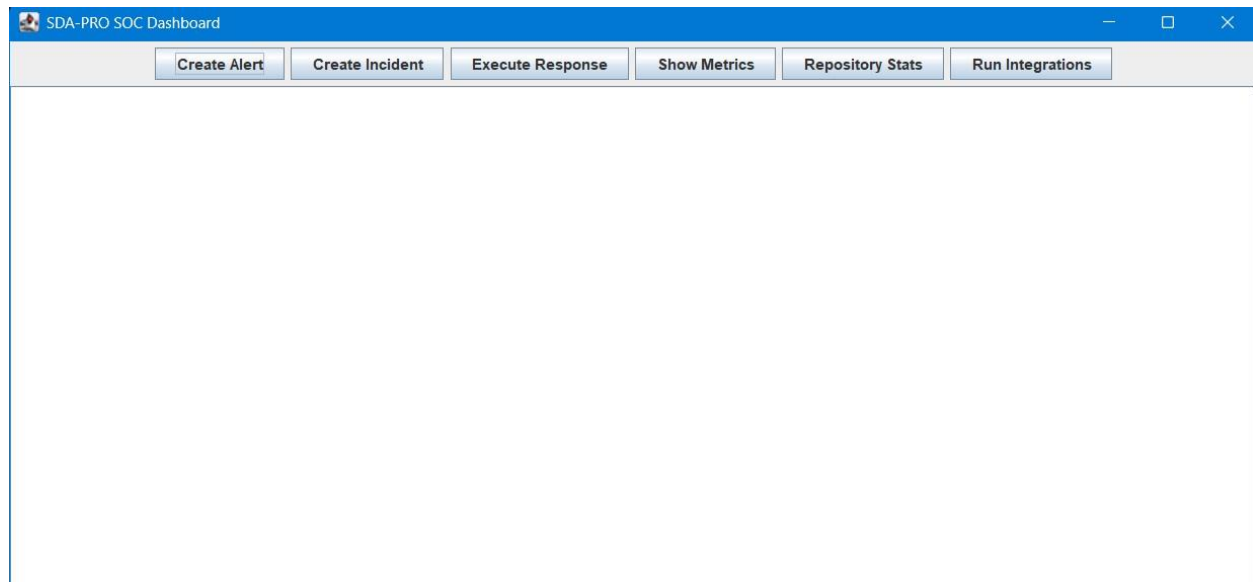
CHAPTER 6 – GUI IMPLEMENTATION

The graphical dashboard was developed using Java Swing.

Features:

- Alert creation
- Incident creation
- Execute response
- Dashboard statistics
- Repository stats
- Integration execution

Main Dashboard



Alert Management



Incident Management

 SDA-PRO SOC Dashboard

Create Alert

Create Incident

Execute R

Firewall Alert Created

Alert Ingested From: Firewall

Threat Intelligence Added To: Firewall Alert Created

Notification Sent: Firewall Alert Created

Audit Logged: Firewall Alert Created

Incident Generated

Response Plan Executed

Response execution

Incident Generated
Response Plan Executed

Dashboard & Repository statistics

 SDA-PRO SOC Dashboard

Create Alert

Create Incident

Execute Response

Show Metrics

Firewall Alert Created

Alert Ingested From: Firewall

Threat Intelligence Added To: Firewall Alert Created

Notification Sent: Firewall Alert Created

Audit Logged: Firewall Alert Created

Incident Generated

Response Plan Executed

===== DASHBOARD METRICS =====

Alerts Processed: 1

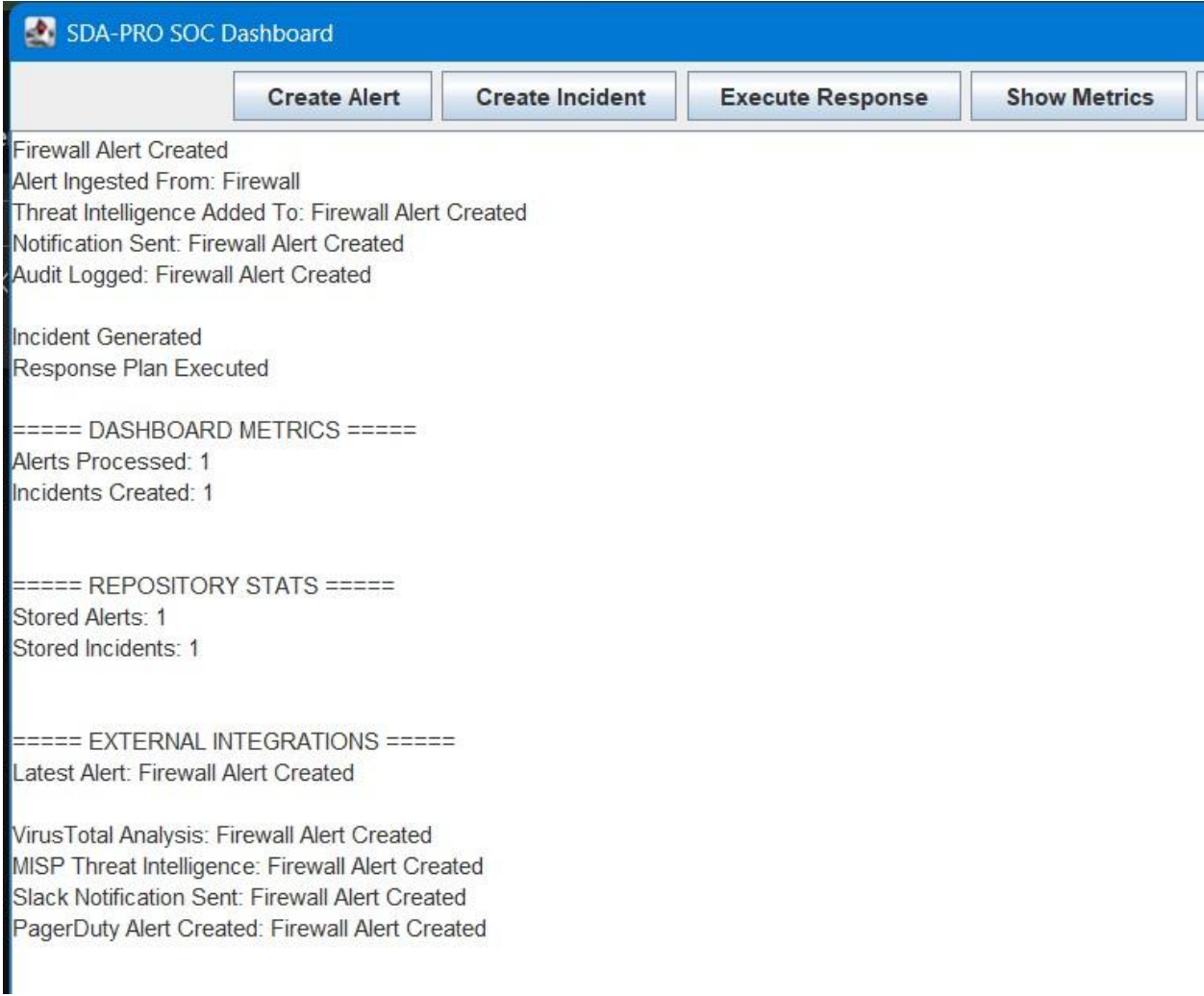
Incidents Created: 1

===== REPOSITORY STATS =====

Stored Alerts: 1

Stored Incidents: 1

Integration execution



The screenshot displays the SDA-PRO SOC Dashboard interface. At the top, a blue header bar contains the dashboard title and a navigation menu with buttons for 'Create Alert', 'Create Incident', 'Execute Response', and 'Show Metrics'. The main content area shows a log of events: 'Firewall Alert Created' followed by 'Alert Ingested From: Firewall', 'Threat Intelligence Added To: Firewall Alert Created', 'Notification Sent: Firewall Alert Created', and 'Audit Logged: Firewall Alert Created'. Below this, it shows 'Incident Generated' and 'Response Plan Executed'. A section titled '===== DASHBOARD METRICS =====' displays 'Alerts Processed: 1' and 'Incidents Created: 1'. Another section titled '===== REPOSITORY STATS =====' shows 'Stored Alerts: 1' and 'Stored Incidents: 1'. A third section titled '===== EXTERNAL INTEGRATIONS =====' lists 'Latest Alert: Firewall Alert Created' and several integration statuses: 'VirusTotal Analysis: Firewall Alert Created', 'MISP Threat Intelligence: Firewall Alert Created', 'Slack Notification Sent: Firewall Alert Created', and 'PagerDuty Alert Created: Firewall Alert Created'.

SDA-PRO SOC Dashboard

Create Alert Create Incident Execute Response Show Metrics

Firewall Alert Created
Alert Ingested From: Firewall
Threat Intelligence Added To: Firewall Alert Created
Notification Sent: Firewall Alert Created
Audit Logged: Firewall Alert Created

Incident Generated
Response Plan Executed

===== DASHBOARD METRICS =====
Alerts Processed: 1
Incidents Created: 1

===== REPOSITORY STATS =====
Stored Alerts: 1
Stored Incidents: 1

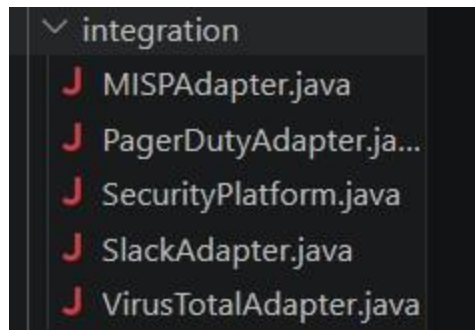
===== EXTERNAL INTEGRATIONS =====
Latest Alert: Firewall Alert Created
VirusTotal Analysis: Firewall Alert Created
MISP Threat Intelligence: Firewall Alert Created
Slack Notification Sent: Firewall Alert Created
PagerDuty Alert Created: Firewall Alert Created

CHAPTER 7 – EXTERNAL INTEGRATIONS

The platform includes simulated integrations with:

- VirusTotal
- MISP
- Slack
- PagerDuty

These integrations are implemented using Adapter Pattern classes.

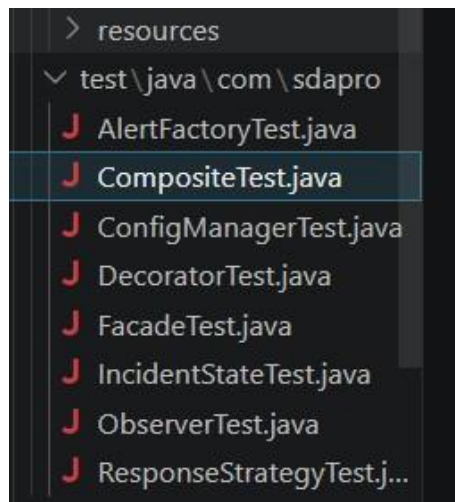


CHAPTER 8 – TESTING AND VALIDATION

JUnit testing was performed to verify functionality.

Implemented Tests:

- AlertFactoryTest
- ConfigManagerTest
- ResponseStrategyTest
- State Pattern Tests
- Observer Pattern Tests
- CompositeTest
- FacadeTest
- IncidentStateTest




```

[INFO] Scanning for projects...
[INFO] -----< sda_pro:sda_pro >-----
[INFO] Building sda_pro 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- resources:3.4.0:resources (default-resources) @ sda_pro ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO] --- compiler:3.15.0:compile (default-compile) @ sda_pro ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- resources:3.4.0:testResources (default-testResources) @ sda_pro ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory A:\MDN\Documents\Programming\sda_pro\src\test\resources
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ sda_pro ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.2.5:test (default-test) @ sda_pro ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.sdapro.AlertFactoryTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.091 s -- in com.sdapro.AlertFactoryTest
[INFO] Running com.sdapro.CompositeTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in com.sdapro.CompositeTest
[INFO] Running com.sdapro.ConfigManagerTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in com.sdapro.ConfigManagerTest
[INFO] Running com.sdapro.DecoratorTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in com.sdapro.DecoratorTest
[INFO] Running com.sdapro.FacadeTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in com.sdapro.FacadeTest
[INFO] Running com.sdapro.IncidentStateTest
Transitioning NEW -> TRIAGE
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in com.sdapro.IncidentStateTest
[INFO] Running com.sdapro.ObserverTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in com.sdapro.ObserverTest
[INFO] Running com.sdapro.ResponseStrategyTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.016 s -- in com.sdapro.ResponseStrategyTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.600 s
[INFO] Finished at: 2026-06-03T22:11:43+05:00
[INFO]

```

Results:

All tests passed successfully.

CHAPTER 9 – DEPLOYMENT

9.1 Maven Build

The application is built using Maven.

Commands:

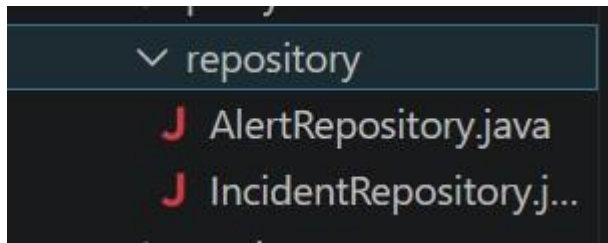
```
mvn clean package mvn
```

```
test
```

9.2 Repository Layer

The system uses:

- AlertRepository
- IncidentRepository

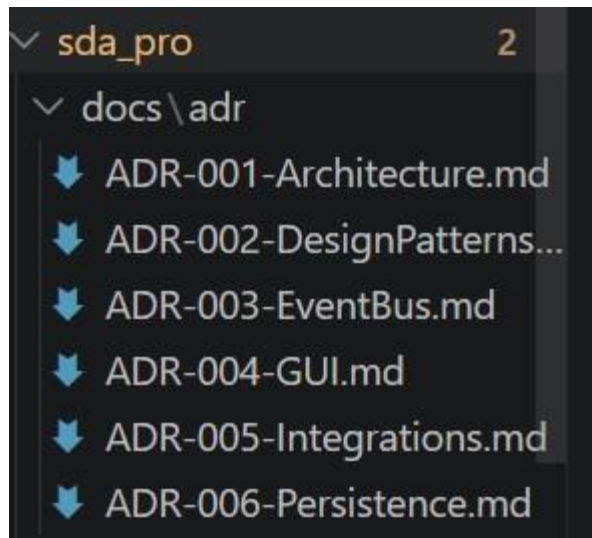


These repositories abstract data storage and support future database migration.

9.3 ADR Documents

Implemented ADRs:

- ADR-001 Architecture
- ADR-002 Design Patterns
- ADR-003 Event Bus
- ADR-004 GUI
- ADR-005 Integrations
- ADR-006 Persistence



CHAPTER 10 – RESULTS AND DISCUSSION

Achievements:

- MVC architecture implemented
- SOA concepts demonstrated
- 12 Design Patterns implemented
- UML diagrams completed
- GUI dashboard completed
- Integration layer completed
- Testing completed

The project successfully demonstrates advanced software design principles within a cybersecurity context.

CHAPTER 11 – CONCLUSION

SDA-PRO successfully achieved all project objectives. The platform demonstrates how design patterns and architectural principles can be combined to create a maintainable and extensible cybersecurity incident response system.

APPENDICES

Appendix A – Source Code Structure

```
src/
├─ main/
│   └─ java/
│       └─ com/sdapro/
│           ├── Main.java
│           ├── abstractfactory/ (6 files)
│           ├── adapter/ (3 files)
│           ├── chain/ (6 files)
│           ├── composite/ (5 files)
│           ├── controller/ (3 files)
│           ├── decorator/ (7 files)
│           ├── eventbus/ (6 files)
│           ├── facade/ (1 file)
│           ├── factory/ (1 file)
│           ├── integration/ (5 files)
│           ├── model/ (2 files)
│           ├── models/ (3 files)
│           ├── normalizer/ (6 files)
│           ├── notification/ (8 files)
│           ├── observer/ (5 files)
│           ├── proxy/ (1 file)
│           ├── repository/ (2 files)
│           ├── service/ (6 files)
│           ├── singleton/ (1 file)
│           ├── state/ (9 files)
│           ├── strategy/ (5 files)
│           └─ ui/ (2 files)
│       └─ resources/
├─ test/
│   └─ java/
│       └─ com/sdapro/
│           ├── AlertFactoryTest.java
│           ├── CompositeTest.java
│           ├── ConfigManagerTest.java
│           ├── DecoratorTest.java
│           ├── FacadeTest.java
│           ├── IncidentStateTest.java
│           ├── ObserverTest.java
│           └─ ResponseStrategyTest.java
```

Appendix B – Test Results

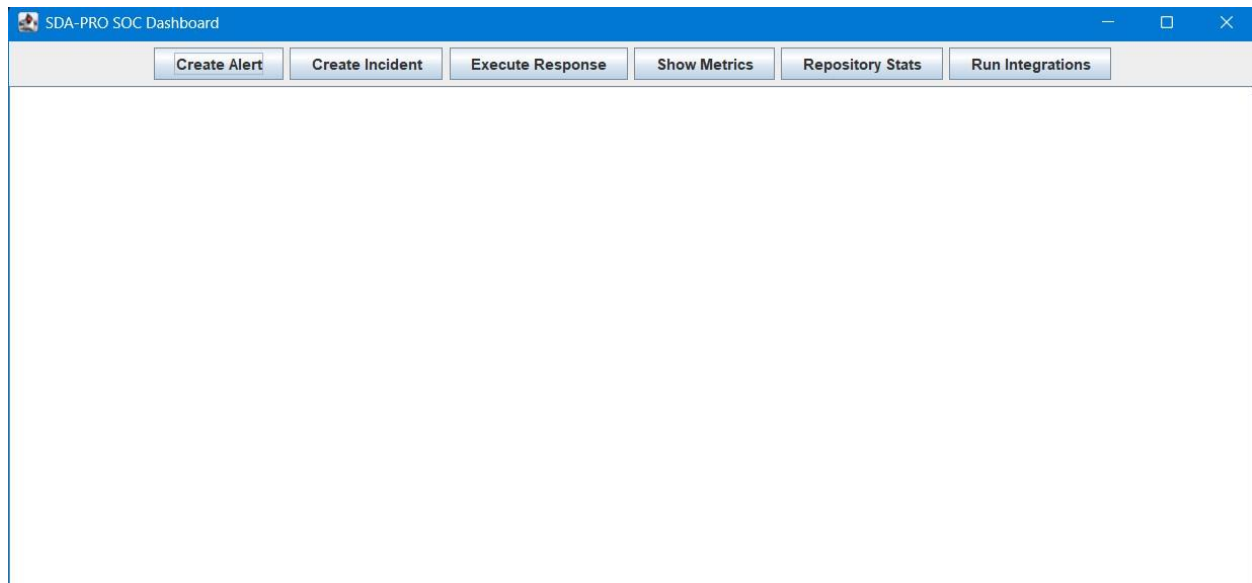
```

[INFO] Scanning for projects...
[INFO]
[INFO] -----< sda_pro:sda_pro >-----
[INFO] Building sda_pro 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ sda_pro ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ sda_pro ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ sda_pro ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory A:\VJN\Documents\Programming\sda_pro\src\test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ sda_pro ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ sda_pro ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.sdapro.AlertFactoryTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.091 s -- in com.sdapro.AlertFactoryTest
[INFO] Running com.sdapro.CompositeTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in com.sdapro.CompositeTest
[INFO] Running com.sdapro.ConfigManagerTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in com.sdapro.ConfigManagerTest
[INFO] Running com.sdapro.DecoratorTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in com.sdapro.DecoratorTest
[INFO] Running com.sdapro.FacadeTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in com.sdapro.FacadeTest
[INFO] Running com.sdapro.IncidentStateTest
Transitioning NEW -> TRIAGE
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in com.sdapro.IncidentStateTest
[INFO] Running com.sdapro.ObserverTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in com.sdapro.ObserverTest
[INFO] Running com.sdapro.ResponseStrategyTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.016 s -- in com.sdapro.ResponseStrategyTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.600 s
[INFO] Finished at: 2026-06-03T22:11:43+05:00
[INFO]

```

Appendix C – GUI Screenshots

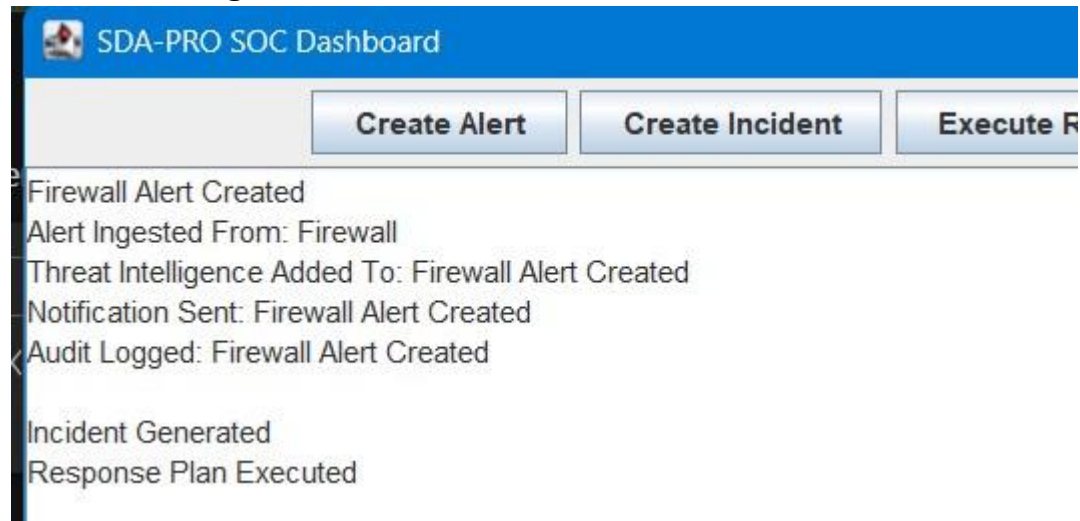
Main Dashboard



Alert Management



Incident Management



Response execution

Incident Generated

Response Plan Executed

Dashboard & Repository statistics

 SDA-PRO SOC Dashboard

[Create Alert](#)[Create Incident](#)[Execute Response](#)[Show Metrics](#)

Firewall Alert Created
Alert Ingested From: Firewall
Threat Intelligence Added To: Firewall Alert Created
Notification Sent: Firewall Alert Created
Audit Logged: Firewall Alert Created

Incident Generated
Response Plan Executed

===== DASHBOARD METRICS =====
Alerts Processed: 1
Incidents Created: 1

===== REPOSITORY STATS =====
Stored Alerts: 1
Stored Incidents: 1

Integration execution

 SDA-PRO SOC Dashboard

[Create Alert](#) [Create Incident](#) [Execute Response](#) [Show Metrics](#)

Firewall Alert Created
Alert Ingested From: Firewall
Threat Intelligence Added To: Firewall Alert Created
Notification Sent: Firewall Alert Created
Audit Logged: Firewall Alert Created

Incident Generated
Response Plan Executed

===== DASHBOARD METRICS =====
Alerts Processed: 1
Incidents Created: 1

===== REPOSITORY STATS =====
Stored Alerts: 1
Stored Incidents: 1

===== EXTERNAL INTEGRATIONS =====
Latest Alert: Firewall Alert Created

VirusTotal Analysis: Firewall Alert Created
MISP Threat Intelligence: Firewall Alert Created
Slack Notification Sent: Firewall Alert Created
PagerDuty Alert Created: Firewall Alert Created

ADR-001: System Architecture

Status

Accepted

Context

SDA-PRO is a Security Detection and Automated Response Platform.

The system requires:

- Separation of concerns
- Easy maintenance
- Extensibility - Modular design

Decision

A layered architecture was selected.

Layers:

1. UI Layer
2. Controller Layer
3. Service Layer
4. Repository Layer
5. Integration Layer

The system also follows MVC principles.

Consequences

Benefits:

- Loose coupling
- Better maintainability
- Easier testing - Scalability

Drawbacks:

- More classes and packages

ADR-002: Design Pattern Selection

Status

Accepted

Context

The project requires reusable and extensible software design.

Decision

The following patterns were implemented:

- Factory Method
- Abstract Factory
- Singleton
- Strategy - State
- Observer
- Chain of Responsibility
- Decorator
- Composite
- Adapter
- Proxy - Facade

Consequences

Benefits:

- Better extensibility
- Reusable components
- Reduced coupling
- Improved maintainability

Drawbacks:

- Increased number of classes

ADR-003: Event Bus Implementation

Status

Accepted

Context

Multiple components need notification when incidents or alerts are created.

Examples:

- Dashboard updates
- Audit logging
- Notifications - Metrics collection

Decision

An EventBusPublisher singleton was implemented using the Observer Pattern.

Observers:

- DashboardUpdater
- AuditEventLogger
- NotificationDispatcher - MetricsCollector

Consequences

Benefits:

- Loose coupling
- Easy addition of new observers
- Better scalability

Drawbacks:

- More event management complexity

ADR-004: GUI Technology Selection

Status

Accepted

Context

The project requires a graphical dashboard.

Decision

Java Swing was selected.

Reasons:

- Included with Java
- Lightweight
- Easy deployment
- No external dependencies

Consequences

Benefits:

- Simple implementation
- Fast development - Platform independent

Drawbacks:

- Less modern than web frameworks

ADR-005: External Integration Strategy

Status

Accepted

Context

The system integrates with external cybersecurity services.

Examples:

- VirusTotal
- MISP
- Slack - PagerDuty

Decision

Adapter Pattern was used to standardize communication.

Adapters:

- VirusTotalAdapter
- MISPAdapter
- SlackAdapter - PagerDutyAdapter

Consequences

Benefits:

- Consistent interfaces
- Easier future integrations - Reduced dependency on vendor APIs

Drawbacks:

- Additional abstraction layer

ADR-006: Persistence Strategy

Status

Accepted

Context

The project requires storage of alerts and incidents.

Decision

Repository Pattern was selected.

Current implementation uses in-memory repositories:

- AlertRepository - IncidentRepository

Future versions may use PostgreSQL or MySQL without changing service logic.

Consequences

Benefits:

- Simplicity
- Fast development - Loose coupling

Drawbacks:

- Data not retained after application shutdown